Alphalens Documentation

Contents

1 Overview **3**

2 alphalens.utils **3**
- 2.1 Data model used by this module . . . 3
- 2.2 Exceptions . . . 3
- 2.3 Core workflow . . . 3
- 2.4 Quantization and bucketing . . . 5
- 2.5 Return and performance helpers . . . 5
- 2.6 Calendar and time utilities . . . 6
- 2.7 Column detection and display helpers . . . 7
- 2.8 Error handling helper . . . 7
- 2.9 Per-function dependency table . . . 8
- 2.10 Practical usage . . . 10
- 2.11 Notes . . . 10

3 alphalens.performance **10**
- 3.1 Data model used by this module . . . 10
- 3.2 Core analysis functions . . . 10
- 3.3 Portfolio weighting and return simulation . . . 11
- 3.4 Quantile and bucket analysis . . . 14
- 3.5 Event-study helpers . . . 16
- 3.6 Portfolio simulation outputs . . . 17
- 3.7 Dependencies and implementation notes . . . 18
- 3.8 Per-function dependency table . . . 18
- 3.9 Practical usage . . . 20

4 alphalens.plotting **20**
- 4.1 Styling and context management . . . 20
- 4.2 Data table displays . . . 21
- 4.3 Information Coefficient visualization . . . 22
- 4.4 Returns analysis . . . 23
- 4.5 Factor characteristics . . . 25
- 4.6 Constants . . . 25
- 4.7 Dependencies and conventions . . . 25
- 4.8 Practical usage . . . 26
- 4.9 Notes . . . 26

5 alphalens.tears **26**
5.1 Helper class 27
5.2 Tear sheet generators 27
5.3 Data model 31
5.4 Dependencies and concepts 31
5.5 Practical usage 31
5.6 Notes 32

1 Overview

Alphalens is organized as a pipeline of four core modules:

- `alphalens.utils` prepares factor data and computes forward returns.
- `alphalens.performance` analyzes factor behavior and simulated portfolio performance.
- `alphalens.plotting` visualizes metrics and diagnostics.
- `alphalens.tears` assembles the results into tear sheets.

The usual workflow is:

1. Build cleaned factor data with `alphalens.utils.get_clean_factor_and_forward_returns()`.
2. Compute diagnostics and portfolio statistics with `alphalens.performance`.
3. Visualize outputs with `alphalens.plotting`.
4. Generate presentation-ready reports with `alphalens.tears`.

All modules typically operate on a MultiIndex `pandas` object indexed by `(date, asset)`.

2 alphalens.utils

Utilities for preparing factor data, computing forward returns, binning factors into quantiles/bins, and working with custom trading calendars.

This module is the main data-preparation layer used throughout Alphalens. Most workflows start with `get_clean_factor_and_forward_returns()` and then feed the resulting MultiIndex `DataFrame` into the tear sheets and plotting functions.

2.1 Data model used by this module

Most functions operate on a `pandas` object indexed by:

- level 0: `date` / timestamp
- level 1: `asset`

Forward returns are usually stored in columns named with `pd.Timedelta`-compatible strings such as `1D`, `5D`, `30m`, or `1D1h`.

2.2 Exceptions

`NonMatchingTimezoneError` Raised when factor and pricing indices have different timezones.

`MaxLossExceededError` Raised by `get_clean_factor()` when the share of dropped factor data exceeds `max_loss`.

2.3 Core workflow

2.3.1 `compute_forward_returns(factor: pd.Series, prices: pd.DataFrame, periods: Sequence[in 5, 10), filter_zscore: Optional[float]=None, cumulative_returns=True) -> pd.DataFrame`

Compute N-period forward returns for each asset in `factor`.

Input expectations

- `factor`: MultiIndex `Series` indexed by `(date, asset)`.
- `prices`: wide `DataFrame` with dates on the index and assets on the columns.
- The price history must extend far enough beyond the factor timestamps to cover the largest requested period.

Behavior

- Validates that factor and price timezones match.
- Infers a trading calendar when the factor index has no frequency.
- Computes forward returns for each requested period.
- Optionally filters out extreme values using `filter_zscore`.
- Names the output columns using a `Timedelta`-style string.

Returns A MultiIndex `DataFrame` with the same `(date, asset)` index as the input factor.

2.3.2 `get_clean_factor(factor: pd.Series, forward_returns: pd.DataFrame, groupby: Optional[Union[pd.Series, dict[Any, Any]]] = None, binning_by_group: bool = False, quantiles: Optional[Union[int, Sequence[float]]] = 5, bins: Optional[Union[int, Sequence[float]]] = None, groupby_labels: Optional[dict[Any, Any]] = None, max_loss: float = 0.35, zero_aware: bool = False) -> pd.DataFrame`

Align a factor series with already-computed forward returns and assign factor quantiles or bins.

Use this when you already have forward returns and do not need to recompute them.

Key behavior

- Joins factor values to forward returns.
- Optionally maps assets to groups.
- Computes factor quantiles or bins using `quantize_factor()`.
- Drops rows with missing data.
- Tracks how much data was lost and raises `MaxLossExceededError` if the loss exceeds `max_loss`.

Returns A cleaned MultiIndex `DataFrame` containing forward return columns, `factor`, optionally `group`, and `factor_quantile`.

2.3.3 `get_clean_factor_and_forward_returns(factor: pd.Series, prices: pd.DataFrame, groupby: Optional[Union[pd.Series, dict[Any, Any]]] = None, binning_by_group: bool = False, quantiles: Optional[Union[int, Sequence[float]]] = 5, bins: Optional[Union[int, Sequence[float]]] = None, periods: Sequence[int] = (1, 5, 10), filter_zscore: Optional[float] = 20, groupby_labels: Optional[dict[Any, Any]] = None, max_loss: float = 0.35, zero_aware: bool = False, cumulative_returns: bool = True) -> pd.DataFrame`

One-stop helper that computes forward returns and then cleans and bins the factor data.

This is the primary entry point for most Alphalens analyses.

Equivalent steps

1. Call `compute_forward_returns()`
2. Call `get_clean_factor()`

Returns A cleaned MultiIndex `DataFrame` suitable for Alphalens tear sheets and plots.

2.4 Quantization and bucketing

2.4.1 `quantize_factor(factor_data: pd.DataFrame, quantiles: Optional[Union[int, Sequence[flo = 5, bins: Optional[Union[int, Sequence[float]]] = None, by_group: bool = False, no_raise: bool = False, zero_aware: bool = False) -> pd.Series`

Assign factor values to quantile or value bins.

Behavior

- Exactly one of `quantiles` or `bins` must be provided.
- Supports group-wise binning with `by_group=True`.
- Supports zero-aware binning, which splits positive and negative values separately.
- Can suppress binning failures with `no_raise=True`, returning `NaN` for problematic rows.

Returns A `Series` named `factor_quantile` indexed by `(date, asset)`.

2.4.2 `non_unique_bin_edges_error(func)`

Decorator that adds a more helpful error message when `pandas` cannot create unique bin edges.

This is used internally around `quantize_factor()` to explain common binning failures, especially when many identical factor values span multiple quantiles.

2.5 Return and performance helpers

2.5.1 `backshift_returns_series(series, N)`

Shift a MultiIndex returns series backward by `N` observations in the first level.

This helper is useful for converting backward-looking returns into forward-looking returns.

2.5.2 `demean_forward_returns(factor_data, grouper=None)`

Demean forward returns by date or by a custom grouper.

Behavior

- If `grouper` is not provided, demeaning happens per date.
- Only forward-return columns are adjusted.
- The result preserves the original shape and index.

2.5.3 `rate_of_return(period_ret, base_period)`

Convert returns observed over one period length into an equivalent rate for `base_period`.

This is useful for normalizing returns across different horizons.

2.5.4 `std_conversion(period_std, base_period)`

Convert standard deviation or standard error from one period length to another.

2.5.5 `ic_autocor_adj(daily_ic: pd.DataFrame, lag: int = 5) -> pd.DataFrame`

Compute IC autocorrelation diagnostics and Newey-West-adjusted t-statistics for daily IC series.

Behavior

- Works column-wise on a DataFrame of daily IC values.
- Computes the lagged autocorrelation, Ljung-Box p-value, Newey-West t-stat, and Newey-West p-value for each horizon.
- Returns a transposed summary table with diagnostics as rows and horizons as columns.
- Used by `plot_information_table()` to add lag-1 diagnostics to IC summary tables.

Returns A `DataFrame` of IC autocorrelation diagnostics rounded to three decimals.

2.6 Calendar and time utilities

2.6.1 `infer_trading_calendar(factor_idx, prices_idx)`

Infer a trading calendar from factor and price datetimes.

Behavior

- Detects which weekdays are traded.
- Infers holidays by comparing the observed timestamps to a weekday-specific business-day calendar.
- Returns a `CustomBusinessDay` offset.

2.6.2 `timedelta_to_string(timedelta)`

Convert a `pd.Timedelta` into a compact string compatible with `pd.Timedelta(...)`.
Example output formats include `1D`, `3h15m`, and `1D1h`.

2.6.3 `timedelta_strings_to_integers(sequence)`

Convert a sequence of timedelta strings into integer day counts.
Example: `['1D', '5D'] -> [1, 5]`.

2.6.4 `add_custom_calendar_timedelta(input, timedelta, freq)`

Add a `Timedelta` to a timestamp or `DatetimeIndex` while respecting a custom trading calendar.

Accepted `freq` values

- `Day`
- `BusinessDay`
- `CustomBusinessDay`

2.6.5 `make_naive_ts(t)`

Return a timezone-naive timestamp.

- If `t` is timezone-aware, it is converted to naive UTC-localized time.
- Otherwise it is localized to `None`.

2.6.6 `diff_custom_calendar_timedeltas(start, end, freq)`

Compute the effective elapsed time between two timestamps under a custom calendar.

This is used when forward-return horizons must respect trading days, weekends, and holidays.

Accepted `freq` values

- any `pandas.tseries.offsets.BaseOffset`
- commonly `Day`, `BusinessDay`, or `CustomBusinessDay`

2.7 Column detection and display helpers

2.7.1 `get_forward_returns_columns(columns: pd.Index, require_exact_day_multiple=False) -> pd.Index`

Identify which columns look like forward-return horizons.

Behavior

- Recognizes `Timedelta`-style labels such as `1D`, `5D`, `30m`, `1D1h`.
- When `require_exact_day_multiple=True`, only exact day multiples are kept.
- Emits a warning if non-day-multiple columns are skipped in that mode.
- Expects `columns` to be a `pandas.Index` and returns a filtered `pandas.Index`.

2.7.2 `print_table(table, name=None, fmt=None)`

Pretty-print a `Series` or `DataFrame`.

Behavior

- Uses rich display output when available.
- Falls back to standard formatted output.
- Temporarily changes `pandas` floating-point formatting when `fmt` is provided.

2.8 Error handling helper

2.8.1 `rethrow(exception, additional_message)`

Re-raise an exception while preserving the original stack trace and appending extra context to the message.

This is used internally to make binning errors easier to diagnose.

2.9 Per-function dependency table

This table lists direct dependencies only (not transitive call chains).

Function	Purpose	Direct deps in `utils.py`	Direct external deps
`rethrow`	Re-raise an exception with an appended message while preserving traceback.	-	-
`non_unique_bin_edges_error`	Decorator that rewrites non-unique-bin-edge errors with clearer guidance.	`rethrow`	-
`quantize_factor`	Assign factor values to quantile/bin buckets by date (optionally by group/zero-aware).	`non_unique_bin_edges_error` (decorator)	`pd.qcut`, `pd.cut`, `pd.concat`, `pd.Series`
`infer_trading_calendar`	Infer a trading calendar (`CustomBusinessDay`) from factor and price indices.	-	`pd.date_range`, `CustomBusinessDay`
`compute_forward_returns`	Compute forward returns aligned to factor timestamps and infer/keep trading frequency.	`infer_trading_calendar`, `diff_custom_calendar_timedeltas`, `timedelta_to_string`	`pd.MultiIndex.from_product`, `pd.Timedelta`, `np.concatenate`, `np.nan`, `mode`
`backshift_returns_series`	Shift a MultiIndex returns series backward by `N` observations on date level.	-	`pd.MultiIndex`, `pd.Series`, `np.array`
`demean_forward_returns`	Demean forward returns by date or by provided groupers.	`get_forward_returns_columns`	-
`print_table`	Pretty-print a `Series`/`DataFrame` with optional temporary float formatting.	-	`pd.DataFrame`, `pd.Series`, `pd.get_option`, `pd.set_option`, `display`
`get_clean_factor`	Merge factor, forward returns, and group labels; then bin and enforce max-loss checks.	`quantize_factor`	`pd.Series`, `np.isfinite`
`get_clean_factor_and_forward_returns`	End-to-end helper to compute forward returns and return clean factor data.	`compute_forward_returns`, `get_clean_factor`	-
`rate_of_return`	Convert period return to an equivalent rate for a base period.	-	`pd.Timedelta`
`std_conversion`	Scale period std/std-error to an equivalent base-period value.	-	`pd.Timedelta`, `np.sqrt`
`get_forward_returns_columns`	Detect columns that match forward-return horizon naming conventions.	-	`re.compile`, `warnings.warn`
`timedelta_to_string`	Convert a `Timedelta` to the string format used in forward-return column names.	-	-
`timedelta_strings_to_integers`	Convert timedelta	-	`pd.Timedelta`

2.10 Practical usage

Typical Alphalens usage looks like this:

1. Prepare a factor series indexed by `(date, asset)`.
2. Prepare a price `DataFrame` with assets in columns.
3. Call `get_clean_factor_and_forward_returns()`.
4. Pass the result into tear sheets or plotting helpers.

Example call: `get_clean_factor_and_forward_returns(factor=..., prices=..., periods=(1, 5, 10))`.

2.11 Notes

- Several functions assume data is already aligned on `(date, asset)`.
- Zero-aware binning is intended for factors centered around zero.
- `filter_zscore` can introduce lookahead bias, so use it carefully.
- Forward-return horizon labels are derived from the price calendar and may reflect custom business-day offsets.

3 alphalens.performance

Performance-analysis utilities for factor research, including information coefficient calculations, factor-weighted portfolio simulation, quantile analysis, event-study helpers, and Pyfolio-ready outputs.

This module is the main analytics layer used after factor data has been cleaned and aligned with `alphalens.utils.get_clean_factor_and_forward_returns()`.

3.1 Data model used by this module

Most functions expect a MultiIndex `pandas` object indexed by:

- level 0: `date`
- level 1: `asset`

The input usually includes:

- a `factor` column
- one or more forward-return columns such as `1D`, `5D`, or `10D`
- `factor_quantile`
- optionally `group`

Forward-return column names are detected using `alphalens.utils.get_forward_returns_columns()`.

3.2 Core analysis functions

3.2.1 `factor_information_coefficient(factor_data, group_adjust=False, by_group=False, ic_type="spearman")`

Compute the Information Coefficient (IC) between factor values and each forward-return horizon using either Spearman rank or Pearson correlation.

Behavior

- Calculates the IC period by period using either Spearman rank correlation (default) or Pearson correlation.
- Optionally demeans forward returns by group before computing IC.
- Optionally computes IC separately for each group.
- Preserves the factor data frequency on the date index when not grouped by asset group.
- Raises an error for unsupported `ic_type` values.

Returns A `DataFrame` of IC values indexed by date, or by date and group when `by_group=True`.

3.2.2 `mean_information_coefficient(factor_data, group_adjust=False, by_group=False, by_time=None)`

Compute the mean IC across the full sample or over a time window.

Behavior

- Delegates to `factor_information_coefficient()` using its default IC type (Spearman); call the lower-level helper directly if you need Pearson IC.
- Can aggregate by time period using a pandas time rule such as monthly or weekly windows.
- Can aggregate by group when `by_group=True`.

Returns A scalar-like `Series`/`DataFrame` of mean IC values depending on the requested grouping.

3.3 Portfolio weighting and return simulation

3.3.1 `factor_weights(factor_data, demeaned=True, group_adjust=False, equal_weight=False)`

Compute asset weights from factor values.

Behavior

- Factor values are normalized to gross leverage of 1.
- When `demeaned=True`, weights are constructed as a long-short portfolio.
- When `group_adjust=True`, weights are computed in a group-neutral way.
- When `equal_weight=True`, assets are equal-weighted instead of factor-weighted.

Returns A `Series` of weights indexed by `(date, asset)`.

Worked example Suppose one date contains three assets with these factor values:

asset	factor
A	2.0
B	1.0
C	-1.0

With the default `demeaned=True`, `group_adjust=False`, and `equal_weight=False`:

1. Demean factor values by date.
   - Mean factor $= (2.0 + 1.0 - 1.0)/3 = 0.6667$
   - Demeaned values = `A: 1.3333`, `B: 0.3333`, `C: -1.6667`
2. Normalize by the sum of absolute values.
   - Gross exposure $= |1.3333| + |0.3333| + |-1.6667| = 3.3333$
   - Weights = `A: 0.40`, `B: 0.10`, `C: -0.50`

These weights represent a dollar-neutral long-short portfolio: long positions (`A: 0.40`, `B: 0.10`) total 0.50 and short positions (`C: -0.50`) total 0.50 in absolute value.

If `demeaned=False`, weights come directly from the raw (non-demeaned) factor values: the sum of absolute raw factors would be $|2.0| + |1.0| + |-1.0| = 4.0$, yielding weights `A: 0.50`, `B: 0.25`, `C: -0.25`. If `group_adjust=True`, weights are adjusted to be group-neutral. If `equal_weight=True`, assets are equal-weighted within long and short buckets instead of using factor magnitudes.

3.3.2 `factor_returns(factor_data, demeaned=True, group_adjust=False, equal_weight=False, by_asset=False)`

Compute period-wise returns for a factor-weighted portfolio.

In this module, a **factor return** is the return of a simulated portfolio whose asset weights are derived from the factor signal. For each date and each forward-return horizon, the return is computed as the sum of:

$$\text{asset weight} \times \text{asset forward return}$$

Equivalently:

$$\text{factor return}[t, \text{period}] = \sum_i w_{t,i} \cdot r_{t,i,\text{period}}$$

Behavior

- Uses `factor_weights()` to build the portfolio.
- Multiplies weights by forward-return columns.
- If `by_asset=True`, returns the weighted return contribution of each asset.
- Otherwise, aggregates to date-level portfolio returns.

Worked example Suppose one date contains three assets with these factor values and `1D` forward returns:

asset	factor	`1D` forward return
A	2.0	0.010
B	1.0	0.020
C	-1.0	-0.010

With the default `demeaned=True`, `group_adjust=False`, and `equal_weight=False`:

1. Demean factor values by date.

- Mean factor = $(2.0 + 1.0 - 1.0)/3 = 0.6667$
- Demeaned values = `A: 1.3333`, `B: 0.3333`, `C: -1.6667`

2. Normalize by the sum of absolute values.
   - Gross exposure = $|1.3333| + |0.3333| + |-1.6667| = 3.3333$
   - Weights = `A: 0.40`, `B: 0.10`, `C: -0.50`
3. Multiply each weight by the forward return.
   - `A: 0.40 × 0.010 = 0.0040`
   - `B: 0.10 × 0.020 = 0.0020`
   - `C: -0.50 × -0.010 = 0.0050`
4. Sum across assets.
   - factor return = 0.0040 + 0.0020 + 0.0050 = 0.0110

So the factor portfolio return for that date and period is 0.011, or 1.1%.

If `demeaned=False`, the same formula is used, but the weights come directly from the raw factor values rather than demeaned values. If `group_adjust=True`, weights are computed in a group-neutral way. If `equal_weight=True`, the code uses equal-weighted long/short buckets instead of factor-magnitude weights.

Returns A `DataFrame` of period returns, or asset-level weighted returns when `by_asset=True`.

3.3.3 `factor_alpha_beta(factor_data: pd.DataFrame, returns: Optional[Union[pd.DataFrame, pd.Series]]=None, demeaned: bool=True, group_adjust: bool=False, equal_weight: bool=False) -> pd.DataFrame`

Estimate alpha and beta for a factor portfolio using OLS regression.

Behavior

- Uses factor portfolio returns as the dependent variable.
- Uses the universe mean return as the explanatory variable.
- If `returns` is not provided, it is computed using `factor_returns()`.
- Annualizes alpha using a 252-trading-day convention.

Returns A `DataFrame` containing at least annualized alpha and beta by forward-return horizon.

How alpha and beta are calculated For each forward-return horizon (for example `1D`, `5D`), the function runs:

$$r_{\text{factor}}[t] = \alpha + \beta \cdot r_{\text{universe}}[t] + \varepsilon[t]$$

- $r_{\text{factor}}[t]$: factor portfolio return from `factor_returns()`.
- $r_{\text{universe}}[t]$: cross-sectional mean universe return for the same horizon.
- β: OLS slope (sensitivity to universe return).
- α: OLS intercept, then annualized as $(1+\alpha)^{\texttt{Timedelta("252Days")/Timedelta(period)}} - 1$.

The output currently stores rows labeled `Ann. alpha` and `beta` per horizon.

3.3.4 `cumulative_returns(returns)`

Convert simple returns into cumulative returns.

This is a thin wrapper around `empyrical.cum_returns()` with a starting value of 1.

3.3.5 `positions(weights, period, freq=None)`

Build a time series of portfolio positions from factor weights.

Behavior

- Treats weights as active for a holding period defined by `period`.
- Uses the provided trading calendar frequency, or infers one from the weight index.
- Falls back to `BDay` and emits a warning if no frequency is available.
- Recomputes portfolio weights as positions roll forward through time.

Returns A `DataFrame` with timestamps on the index and assets on the columns.

3.4 Quantile and bucket analysis

3.3.1 `mean_return_by_quantile(factor_data, by_date=False, by_group=False, demeaned=True, group_adjust=False)`

Compute mean forward returns and standard errors by factor quantile.

Behavior

- Can compute results by date or across the full sample.
- Can compute results by group.
- Can demean by the whole universe or within each group.

Returns A pair: (mean_ret, std_error_ret).

Index shape by (`by_date`, `by_group`) For both outputs (`mean_ret` and `std_error_ret`), columns are forward-return horizons (for example `1D`, `5D`, `10D`).

- `by_date=True`, `by_group=False`: index is `(factor_quantile, date)`.
- `by_date=True`, `by_group=True`: index is `(factor_quantile, date, group)`.
- `by_date=False`, `by_group=False`: index is `(factor_quantile)`.
- `by_date=False`, `by_group=True`: index is `(factor_quantile, group)`.

When `by_date=False`, the function first computes date-level statistics and then averages over the date level.

Numerical illustration Assume `by_date=False`, `by_group=False`, and one horizon (`1D`).

- Date-level quantile means after the first aggregation step:
  - Q1: `[-0.010, -0.006]`
  - Q5: `[0.012, 0.008]`
- Final mean by quantile (second aggregation across dates):

  - Q1 mean: `(-0.010 + -0.006) / 2 = -0.008`
  - Q5 mean: `(0.012 + 0.008) / 2 = 0.010`
- Standard error uses `std / sqrt(count)` on those date-level means:
  - For both Q1 and Q5, sample std is about `0.002828` and `count=2`, so
  - `std_error_ret ~ 0.002828 / sqrt(2) = 0.002`.

3.4.2 `compute_mean_returns_spread(mean_returns, upper_quant, lower_quant, std_err=None)`

Compute the difference in mean returns between two quantiles.

Behavior

- Subtracts lower-quantile mean returns from upper-quantile mean returns.
- Optionally propagates standard error for the spread.

Returns A pair: (mean_return_difference, joint_std_err).

3.4.3 `quantile_turnover(quantile_factor, quantile, period=1)`

Measure the proportion of names that leave a given quantile over time.

Behavior

- Compares membership in the selected quantile against the prior period.
- Preserves the date frequency of the input.

Worked example Suppose we evaluate `quantile=5` with `period=1`, and the names in quantile 5 are:

date	names in quantile 5
2026-01-02	$\{A, B, C\}$
2026-01-03	$\{B, C, D\}$
2026-01-06	$\{C, D, E\}$

For each date after the first, turnover is:

$$\frac{\#\text{ new names vs previous date}}{\text{current quantile size}}$$

- On `2026-01-03`, new names vs `2026-01-02` are $\{D\}$, so turnover is $1/3 = 0.3333$.
- On `2026-01-06`, new names vs `2026-01-03` are $\{E\}$, so turnover is $1/3 = 0.3333$.

The first date has no prior comparison point, so no turnover value is reported for it.

Companion example (`period=2`) Using the same memberships, each date is compared to the set two dates earlier.

- On `2026-01-06`, compare $\{C, D, E\}$ to `2026-01-02` ($\{A, B, C\}$).
- New names are $\{D, E\}$, so turnover is $2/3 = 0.6667$.

This highlights that larger `period` values measure non-adjacent membership change. If intermediate dates are missing after frequency alignment (for example via `asfreq`), larger `period` comparisons can be skipped or shifted to different comparable dates.

Returns A `Series` indexed by date.

3.4.4 `factor_rank_autocorrelation(factor_data, period=1)`

Measure the autocorrelation of factor ranks across periods.

Behavior

- Ranks assets by their factor values within each date, then computes the correlation between each date's rank vector and the rank vector period dates earlier.
- Useful as a turnover/stability diagnostic.

Worked example (`period=1`) Suppose the factor values for assets `A`, `B`, `C`, `D` are:

date	factor values (A, B, C, D)	rank vector (A, B, C, D)
2026-01-02	$(1, 2, 3, 4)$	$(1, 2, 3, 4)$
2026-01-03	$(4, 3, 2, 1)$	$(4, 3, 2, 1)$
2026-01-06	$(1, 2, 3, 4)$	$(1, 2, 3, 4)$

With `period=1`, each date is correlated with the immediately prior date:

- On `2026-01-03`: $\text{corr}(4, 3, 2, 1), (1, 2, 3, 4) = -1.0$.
- On `2026-01-06`: $\text{corr}(1, 2, 3, 4), (4, 3, 2, 1) = -1.0$.

The first date has no prior rank vector, so its value is `NaN`.

Companion example (`period=2`) Using the same data with `period=2`, `2026-01-06` is compared to `2026-01-02`:

- $\text{corr}(1, 2, 3, 4), (1, 2, 3, 4) = 1.0$.

So a larger `period` can reveal longer-horizon rank stability even when adjacent dates are unstable.

Returns A `Series` of autocorrelation values indexed by date.

3.5 Event-study helpers

3.5.1 `common_start_returns(factor, returns, before, after, cumulative=False, mean_by_date=Fals demean_by=None)`

Align return windows around common event dates.

Behavior

- Builds a return window around each factor date and asset pair.
- Aligns all windows to a common event-time index.
- Can work with cumulative or period returns.
- Can de-mean against a reference universe.
- Can average across assets by date.

Returns A `DataFrame` of aligned return windows.

3.5.2 `average_cumulative_return_by_quantile(factor_data, returns, periods_before=10, periods_after=15, demeaned=True, group_adjust=False, by_group=False)`

Compute average cumulative returns around factor events by quantile.

Behavior

- Uses `common_start_returns()` internally.
- Computes mean and standard deviation of event-time cumulative returns.
- Can separate results by group.
- Supports group-neutral and universe-demeaned variants.

Returns A MultiIndex `DataFrame` containing mean and standard deviation across the event window.

3.6 Portfolio simulation outputs

3.6.1 `factor_cumulative_returns(factor_data, period, long_short=True, group_neutral=False, equal_weight=False, quantiles=None, groups=None)`

Simulate a factor portfolio and return cumulative performance.

Behavior

- Filters to a single forward-return horizon specified by `period`.
- Can limit analysis to selected quantiles or groups.
- Uses `factor_returns()` and then converts the result to cumulative returns.

This means cumulative factor returns are built directly from the period-wise factor return series described above.

Returns A cumulative return `Series`.

3.6.2 `factor_positions(factor_data, period, long_short=True, group_neutral=False, equal_weight quantiles=None, groups=None)`

Simulate a factor portfolio and return the time series of positions.

Behavior

- Filters to the requested forward-return horizon.
- Reuses `factor_weights()` to compute holdings.
- Converts weights into rolling positions with `positions()`.

Returns A `DataFrame` of asset positions over time.

3.6.3 `create_pyfolio_input(factor_data, period, capital=None, long_short=True, group_neutral=F equal_weight=False, quantiles=None, groups=None, benchmark_period="1D")`

Create returns, positions, and benchmark data in the format expected by Pyfolio.

Behavior

- Builds cumulative strategy returns.
- Resamples returns and positions to daily frequency.
- Adds a `cash` column to the positions output.
- Optionally converts percentage positions into dollar positions using `capital`.
- Computes a benchmark series from the factor universe when the benchmark period is available.

Returns A tuple: (returns, positions, benchmark).

3.7 Dependencies and implementation notes

- Uses `pandas`, `numpy`, `scipy.stats`, `statsmodels`, and `empyrical`.
- Relies heavily on `alphalens.utils` for forward-return column detection and return demeaning.
- Most functions assume data has already been cleaned and aligned into the Alphalens MultiIndex format.
- Several functions use a trading-calendar frequency attached to the date index, so preserving index frequency is important for correct behavior.

3.8 Per-function dependency table

Function	Purpose	Direct deps in `performance.py`	Direct deps in `utils`	Direct external deps
`factor_information_coefficient`	Compute daily IC (Spearman/Pearson) between factor and forward returns.	-	`get_forward_returns_columns`, `demean_forward_returns`	`scipy.stats.spearmanr`, `scipy.stats.pearsonr`
`mean_information_coefficient`	Aggregate IC means by time and/or group.	`factor_information_coefficient`		`pd.Grouper`
`factor_weights`	Build normalized factor-based asset weights (optional demean/group adjust/equal-weight).	-	-	-
`factor_returns`	Compute factor-weighted forward returns per period.	`factor_weights`	`get_forward_returns_columns`	
`factor_alpha_beta`	Regress factor returns on universe mean returns to estimate alpha/beta.	`factor_returns`	`get_forward_returns_columns`	`add_constant`, `OLS`
`cumulative_returns`	Convert simple returns series to cumulative returns.	-	-	`ep.cum_returns`
`positions`	Build position time series from weights and holding periods.	-	`add_custom_calendar_timedelta`	`BDay`, `warnings.warn`
`mean_return_by_quantile`	Compute mean/std-error of forward returns by quantile/date/group.	-	`demean_forward_returns`, `get_forward_returns_columns`	`np.sqrt`
`compute_mean_returns_spread`	Compute spread between upper and lower quantile mean returns.	-	-	`np.sqrt`
`quantile_turnover`	Measure membership turnover for a quantile over a lag period.	-	-	`pd.isna`
`factor_rank_autocorrelation`	Compute autocorrelation of mean factor rank across dates.	-	-	-
`common_start_returns`	Align returns around event dates into common-relative windows.	`cumulative_returns`	-	-
`average_cumulative_return_by_quantile`	Compute average cumulative event returns by quantile (optionally by group).	`common_start_returns`	-	`np.inf`
`factor_cumulative_returns`	Simulate portfolio and return cumulative returns for a chosen period.	`factor_returns`, `cumulative_returns`	`get_forward_returns_columns`	
`factor_positions`	Simulate portfolio	`factor_weights`,	`get_forward_returns_columns`	

3.9 Practical usage

Typical workflow:

1. Build cleaned factor data with `alphalens.utils.get_clean_factor_and_forward_returns()`.
2. Compute diagnostics such as IC, mean returns by quantile, and turnover.
3. Simulate factor-weighted returns or positions.
4. Feed the outputs into plotting or Pyfolio workflows.

Example call: `factor_information_coefficient(factor_data)` or `create_pyfolio_input(factor_data, period="1D")`.

4 alphalens.plotting

Visualization utilities for factor research, including plots for Information Coefficient analysis, factor returns, quantile performance, and other diagnostic charts. The module provides publication-quality visualizations with sane defaults for the Alphalens workflow.

This module visualizes the outputs from `alphalens.performance` and the cleaned factor data from `alphalens.utils`.

4.1 Styling and context management

4.1.1 `customize(func)` — decorator

Apply default plotting context and style to a function.

Behavior

- Sets seaborn colorblind-friendly palette.
- Applies default plotting context and axes style.
- Can be disabled with `set_context=False` in the decorated function call.

4.1.2 `plotting_context(context="notebook", font_scale=1.5, rc=None)`

Create an alphalens-specific plotting context.

Behavior

- Wraps `seaborn.plotting_context()` with alphalens defaults.
- Default line width is 1.5.
- Can be used as a context manager for multiple plots.

Returns A seaborn plotting context object suitable for use in `with` statements.

4.1.3 `axes_style(style="darkgrid", rc=None)`

Create an alphalens-specific axes style context.

Behavior

- Wraps `seaborn.axes_style()` with alphalens defaults.
- Commonly used with `plotting_context()`.

Returns A seaborn axes style object suitable for use in `with` statements.

4.2 Data table displays

4.2.1 `plot_returns_table(alpha_beta, mean_ret_quantile, mean_ret_spread_quantile, tables_as_fi ax=None)`

Summarize portfolio alpha/beta and quantile returns in basis points.

Behavior

- Combines alpha/beta results with quantile return statistics.
- Converts decimal returns to basis points.
- Renders the table as a figure when `tables_as_figures=True`; otherwise prints text.

Returns A summary `DataFrame`.

4.2.2 `plot_turnover_table(autocorrelation_data, quantile_turnover, tables_as_figures=True, ax=None)`

Summarize mean quantile turnover and factor rank autocorrelation.

Behavior

- Shows mean turnover for each quantile by period.
- Shows mean factor rank autocorrelation.
- Renders the table as a figure when `tables_as_figures=True`; otherwise prints text tables.

Returns Two summary `DataFrame`s: turnover and autocorrelation.

4.2.3 `plot_information_table(ic_data, tables_as_figures=True, ic_type="spearman")`

Display Information Coefficient summary statistics.

Behavior

- Computes IC mean, standard deviation, risk-adjusted IC, and t-statistics.
- Labels the mean row as `IC Mean (spearman)` or `IC Mean (pearson)` depending on `ic_type`.
- Includes skewness, kurtosis, and lag-1 autocorrelation diagnostics from `utils.ic_autocor_adj(ic_data, lag=1)`.
- Renders table figures or prints text tables depending on `tables_as_figures`.

Returns Pretty-printed table or a `DataFrame` when `return_df=True`.

4.2.4 `plot_quantile_statistics_table(factor_data, return_df=False)`

Display factor value statistics by quantile.

Behavior

- Groups factor data by `factor_quantile`.
- Reports min, max, mean, standard deviation, and count per quantile.
- Includes count percentages.

Returns Pretty-printed table or a `DataFrame` when `return_df=True`.

4.3 Information Coefficient visualization

4.3.1 `plot_ic_ts(ic, ax=None)`

Plot the Information Coefficient time series with a rolling 22-day average.

Behavior

- Creates one subplot per forward-return period.
- Overlays daily IC and a 22-day moving average.
- Includes mean and standard deviation annotations.
- Aligns y-axis ranges across subplots.

Returns A numpy array of matplotlib axes.

4.3.2 `plot_ic_hist(ic, ax=None)`

Plot IC as a histogram with kernel density estimate.

Behavior

- Creates a 3-column grid of histograms, one per forward-return period.
- Includes KDE overlay and a vertical line at the mean.
- Clamps x-axis to `[-1, 1]`.

Returns A numpy array of matplotlib axes.

4.3.3 `plot_ic_qq(ic, theoretical_dist=scipy.stats.norm, ax=None)`

Plot IC values against a theoretical distribution using a Q-Q plot.

Behavior

- Creates a 3-column grid of Q-Q plots.
- Supports normal and Student-t distributions.
- Uses `statsmodels.qqplot()` for plotting.

Returns A numpy array of matplotlib axes.

4.3.4 `plot_ic_by_group(ic_group, ax=None)`

Plot IC by group as a bar chart.

Behavior

- Shows mean IC for each group.
- Rotates x-axis labels for readability.

Returns A matplotlib axes object.

4.3.5 `plot_monthly_ic_heatmap(mean_monthly_ic, ax=None)`

Plot IC or returns as a month/year heatmap with color coding.

Behavior

- Creates one heatmap per forward-return period.
- Uses reversed coolwarm colormap (red = negative, blue = positive).
- Annotates cell values.
- Centered at 0.

Returns A numpy array of matplotlib axes.

4.4 Returns analysis

4.4.1 `plot_quantile_returns_bar(mean_ret_by_q, by_group=False, ylim_percentiles=None, ax=None)`

Plot mean returns by factor quantile as a bar chart.

Behavior

- Optionally creates separate subplots for each group.
- Can limit y-axis range using percentiles of observed data.
- Converts to basis points.

Returns A matplotlib axes object or array depending on grouping.

4.4.2 `plot_quantile_returns_violin(return_by_q, ylim_percentiles=None, ax=None)`

Plot return distributions by factor quantile using violin plots.

Behavior

- Shows distribution of returns within each quantile across periods.
- Uses seaborn violin plot with quartile indicators.
- Optionally limits y-axis range using percentiles.

Returns A matplotlib axes object.

4.4.3 `plot_mean_quantile_returns_spread_time_series(mean_returns_spread, std_err=None, bandwidth=1, ax=None)`

Plot the top-minus-bottom quantile return spread over time.

Behavior

- Shows the difference between top and bottom quantile returns period by period.
- Overlays a 22-day rolling average.
- Optionally adds confidence bands based on standard error.
- Symmetrizes y-axis around zero at the 95th percentile.

Returns A matplotlib axes object or array if input is a `DataFrame`.

4.4.4 `plot_cumulative_returns(factor_returns, period, freq=None, title=None, ax=None)`

Plot cumulative returns of a factor-weighted portfolio.

Behavior

- Converts simple returns to cumulative using `performance.cumulative_returns()`.
- Includes a reference line at 1.0 (break-even).
- Displays the forward-return period in the title.

Returns A matplotlib axes object.

4.4.5 `plot_cumulative_returns_by_quantile(quantile_returns, period, freq=None, ax=None)`

Plot cumulative returns separately for each factor quantile.

Behavior

- Uses reversed coolwarm colormap for quantile coloring.
- Uses symmetric log scale on the y-axis for readability across wide ranges.
- Shows all quantiles on a single plot with a legend.

Returns A matplotlib axes object.

4.4.6 `plot_quantile_average_cumulative_return(avg_cumulative_returns, by_quantile=False, std_bar=False, title=None, ax=None)`

Plot average cumulative returns by factor quantile around event dates.

Behavior

- Shows mean cumulative returns across the event window for each quantile.
- Optionally creates separate subplots per quantile for clarity.
- Can overlay standard deviation error bars.
- Uses reversed coolwarm colormap for consistency.
- Marks event date (x=0) with a vertical dashed line.

Returns A matplotlib axes object or array.

4.5 Factor characteristics

4.5.1 `plot_factor_rank_auto_correlation(factor_autocorrelation, period=1, ax=None)`

Plot factor rank autocorrelation over time.

Behavior

- Shows the stability of relative factor rankings.
- Includes a reference line at 0 (no autocorrelation).
- Annotates the mean autocorrelation in a text box.

Returns A matplotlib axes object.

4.5.2 `plot_top_bottom_quantile_turnover(quantile_turnover, period=1, ax=None)`

Plot turnover for the top and bottom quantiles over time.

Behavior

- Extracts the maximum and minimum quantiles.
- Shows the proportion of names new to each quantile each period.
- Useful for detecting data quality or stationarity issues.

Returns A matplotlib axes object.

4.5.3 `plot_events_distribution(events, num_bars=50, ax=None)`

Plot the distribution of factor events across time.

Behavior

- Divides the time span into `num_bars` intervals.
- Counts events in each interval.
- Displays as a bar chart.

Returns A matplotlib axes object.

4.6 Constants

4.6.1 `DECIMAL_TO_BPS = 10000`

Conversion factor for decimal returns to basis points. Used throughout the module when displaying returns-related plots.

4.7 Dependencies and conventions

- **Visualization**: seaborn, matplotlib, scipy.stats
- **Analysis**: pandas, numpy, statsmodels
- **Internal**: `alphalens.utils`, `alphalens.performance`

Color conventions

- **Coolwarm reversed**: Red represents low/negative quantiles; blue represents high/positive quantiles.
- **Colorblind palette**: All plots use seaborn's colorblind palette for accessibility.
- **Line colors**: Green (forestgreen) for means and averages; blue (steelblue) for confidence bands; red (orangered) for rolling averages.

Plot conventions

- **Multi-subplot layouts**: Plots with multiple periods or groups create grid layouts (usually 1×3 or 2×2).
- **Shared axes**: Where applicable, y-axes are shared across subplots to aid comparison.
- **Reference lines**: Zero lines are included on relevant plots to indicate break-even or neutral positions.
- **Log scales**: Cumulative returns plots use symmetric log scale (`symlog`) to handle very large or very small returns.
- **Annotations**: Statistical summaries (mean, std dev) are included in text boxes on many plots.

4.8 Practical usage

Typical workflow:

1. Compute factor diagnostics with `alphalens.performance` functions.
2. Call plotting functions with the results.
3. All plotting functions are decorated with `@customize` to apply consistent styling.
4. Optional: use `set_context=False` in any plotting call to skip automatic styling.

Example call: `plot_ic_ts(ic_data)` or `plot_quantile_returns_bar(mean_ret_by_q)`.

4.9 Notes

- All plotting functions accept an optional `ax` parameter for custom subplot layouts.
- Return values vary: some functions print tables directly, others return `DataFrame`s or axes.
- The `return_df=False` parameter on table functions controls printing vs. returning.
- Most functions are designed for Jupyter notebooks but work in any matplotlib environment.

5 alphalens.tears

Tear sheet generation for comprehensive factor analysis. A tear sheet is a multi-plot report that visualizes factor performance from multiple perspectives, including returns analysis, information coefficient statistics, turnover metrics, and event studies.

This module orchestrates the workflows of `alphalens.performance` and `alphalens.plotting` to produce production-quality analysis reports suitable for presentation and decision-making.

5.1 Helper class

5.1.1 `GridFigure`

Utility class for managing matplotlib grid-based subplot layouts with automatic row/column navigation.

Methods

- `__init__(rows, cols)` — Create a grid figure with specified dimensions.
- `next_row()` — Get the next full-width subplot (advances to the next row).
- `next_cell()` — Get the next cell in the current row (auto-wraps to next row when full).
- `close()` — Close the figure and clean up resources.

Behavior

- Default figure size is 14 inches wide by 7 inches per row.
- Horizontal and vertical spacing are set to 0.4 and 0.3 respectively for readability.

5.2 Tear sheet generators

All tear sheet functions are decorated with `@plotting.customize` to apply consistent styling automatically. The `set_context=False` parameter can be passed to any tear sheet to skip the decorator.

5.2.1 `create_summary_tear_sheet(factor_data, long_short=True, group_neutral=False, tables_as_figures=True)`

Generate a compact tear sheet with key returns, information, and turnover statistics.

Behavior

- Displays factor quantile statistics table.
- Computes and shows alpha, beta, top/bottom quantile returns, and spreads.
- Plots factor quantile returns bar chart.
- Computes and displays IC summary statistics.
- Analyzes turnover and factor rank autocorrelation.
- All metrics respect the `long_short` and `group_neutral` flags.

Returns Displays a matplotlib figure with the complete summary.

5.2.2 `create_returns_tear_sheet(factor_data, long_short=True, group_neutral=False, by_group=False, tables_as_figures=True, save_file=None)`

Detailed tear sheet focused on returns analysis by factor quantile.

Behavior

- Computes factor portfolio returns.
- Displays alpha and beta metrics.
- Shows mean quantile returns as a bar chart.

- Plots return distributions using violin plots (by date across periods).
- If `'1D'` returns are available, plots cumulative portfolio returns and quantile-level cumulative returns.
- Displays top-minus-bottom quantile return spread over time with optional error bands.
- When `by_group=True`, creates separate bar charts for each asset group.
- When `save_file` is set, writes all generated figures into one PDF or image file.

Parameters

- `long_short` — Enable long-short demeaning for dollar-neutral analysis.
- `group_neutral` — Normalize weights and returns across groups.
- `by_group` — Create separate visualizations per group.
- `tables_as_figures` — Render summary tables in the tear sheet when `True` (default); otherwise display text tables and omit table figures.
- `save_file` — Output path for saving all generated plots.

Returns Always returns the returns summary `DataFrame`, independently of table display.

5.2.3 `create_information_tear_sheet(factor_data, group_neutral=False, by_group=False, tables_as_figures=True, save_file=None)`

Tear sheet focused on Information Coefficient (IC) analysis.

Behavior

- Displays both Spearman and Pearson IC summary statistics (mean, std, risk-adjusted IC, t-stat, p-value, skewness, kurtosis).
- Plots IC time series with 22-day rolling average for each forward-return period.
- Plots IC histograms with KDE.
- Plots IC Q-Q plots against normal distribution.
- When `by_group=False` (default), creates a heatmap of monthly mean IC.
- When `by_group=True`, shows mean IC separately for each group.
- When `save_file` is set, saves all generated figures into one file (`*.pdf` multipage, otherwise stitched image).

Parameters

- `group_neutral` — Demean forward returns by group before computing IC.
- `by_group` — Separate analysis by asset group.
- `tables_as_figures` — Render summary tables in the tear sheet when `True` (default); otherwise display text tables and omit table figures.
- `save_file` — Output path for saving generated plots.

Returns Always returns a dictionary of summary DataFrames keyed by `"spearman"` and `"pearson"`, independently of table display.

5.2.4 `create_turnover_tear_sheet(factor_data, turnover_periods=None, tables_as_figures=True, save_file=None)`

Tear sheet analyzing factor turnover and rank stability.

Behavior

- Displays quantile turnover statistics (mean turnover per quantile by period).
- Displays factor rank autocorrelation statistics.
- Plots time series of top and bottom quantile turnover for each period.
- Plots factor rank autocorrelation over time for each period.
- Automatically extracts day-multiple forward-return periods if `turnover_periods` is not provided.
- Supports `save_file` output (`*.pdf` or image via matplotlib save).

Parameters

- `turnover_periods` — Custom periods for turnover analysis. If not provided, uses exact day multiples from `factor_data`.
- `tables_as_figures` — Render summary tables in the tear sheet when `True` (default); otherwise display text tables and omit table figures.
- `save_file` — Output path for saving generated plots.

Returns Always returns `(turnover_table, auto_corr)` summary DataFrames, independently of table display.

5.2.5 `create_full_tear_sheet(factor_data, long_short=True, group_neutral=False, by_group=False, turnover_period=1, period_unit="D", tables_as_figures=True)`

Comprehensive tear sheet combining all analysis perspectives.

Behavior

- Displays factor quantile statistics.
- Calls `create_returns_tear_sheet()` with all parameters passed through.
- Calls `create_information_tear_sheet()` with appropriate flags.
- Calls `create_turnover_tear_sheet()` with defaults.
- All component tear sheets skip automatic styling via `set_context=False` to apply consistent styling once.

Parameters

- `long_short` — Enable long-short demeaning.
- `group_neutral` — Enable group-neutral normalization.
- `by_group` — Separate analysis per group.
- `turnover_period` — Number of periods for turnover analysis.
- `period_unit` — Unit of time for turnover analysis.
- `tables_as_figures` — Render summary tables in the tear sheet when `True` (default); otherwise display text tables and omit table figures.

Returns Displays multiple matplotlib figures for all analyses.

5.2.6 `create_event_returns_tear_sheet(factor_data, returns, avgretplot=(5, 15), long_short=Tru group_neutral=False, std_bar=True, by_group=False)`

Tear sheet analyzing average cumulative returns around factor events (event study).

Behavior

- Computes average cumulative returns within a window before and after each factor event.
- Plots overall average cumulative returns with optional error bands.
- When `std_bar=True`, plots separate cumulative return traces for each quantile with standard deviation error bars.
- When `by_group=True`, repeats the analysis separately for each asset group.
- Useful for analyzing the predictive power of a factor over different time horizons relative to the event date.

Parameters

- `factor_data` — MultiIndex factor data (no forward returns required for this tear sheet).
- `returns` — DataFrame of asset returns indexed by date.
- `avgretplot` — Tuple of `(periods_before, periods_after)` for the event window.
- `long_short` — Demean cumulative returns.
- `group_neutral` — Normalize returns by group.
- `std_bar` — Include error bar plots by quantile.
- `by_group` — Separate analysis per group.

Returns Displays matplotlib figure(s) showing event-window average returns.

5.2.7 `create_event_study_tear_sheet(factor_data, returns, avgretplot=(5, 15), rate_of_ret=True n_bars=50)`

Tear sheet for analyzing specific events or signals.

Behavior

- Displays factor quantile statistics.
- Plots histogram of event distribution across time (divided into `n_bars` intervals).
- Calls `create_event_returns_tear_sheet()` internally for average cumulative returns analysis.
- Computes and displays mean quantile returns.
- Plots return distributions via violin plots by quantile.
- Can optionally annualize returns using `rate_of_ret=True`.

Parameters

- `factor_data` — MultiIndex factor data indexed by event dates.
- `returns` — Asset returns DataFrame.

- `avgretplot` — Window for event-study cumulative returns.
- `rate_of_ret` — Convert simple returns to annualized rates.
- `n_bars` — Number of time intervals for event distribution histogram.

Returns Displays matplotlib figures for event analysis.

5.3 Data model

All tear sheet functions expect `factor_data` to be a MultiIndex `DataFrame` with:

- Index: `(date, asset)`
- Columns:
  - Forward-return columns (e.g., `1D`, `5D`, `10D`)
  - `factor` — The factor values
  - `factor_quantile` — Quantile assignments
  - `group` (optional) — Asset group identifiers

This is the standard output format from `alphalens.utils.get_clean_factor_and_forward_returns()`.

5.4 Dependencies and concepts

- **Visualization**: matplotlib, seaborn
- **Analysis**: pandas, numpy, scipy.stats, statsmodels
- **Internal**: `alphalens.plotting`, `alphalens.performance`, `alphalens.utils`

Key tear sheet concepts

- **Long-short analysis**: Factor values are demeaned, creating equal-weight long and short sides.
- **Group-neutral analysis**: Asset groups are weighted equally, and returns are normalized within groups.
- **Quantile analysis**: Assets are divided into quantiles/buckets based on factor values; returns and metrics are computed per quantile.
- **Information Coefficient**: Spearman rank correlation between factor values and forward returns; higher IC indicates stronger predictive power.
- **Turnover**: Proportion of assets entering/leaving a quantile each period; low turnover suggests stable rankings.
- **Event study**: Analysis of returns in a window around specific factor events or signals.

5.5 Practical usage

Typical workflow:

1. Prepare cleaned factor data with `alphalens.utils.get_clean_factor_and_forward_returns()`.
2. Call the desired tear sheet function.
3. For a quick overview, use `create_summary_tear_sheet()`.
4. For detailed analysis, use `create_full_tear_sheet()`.
5. For event-based analysis, use `create_event_returns_tear_sheet()` or `create_event_study_tear_sheet(`

Example:

```
from alphalens.tears import create_full_tear_sheet

create_full_tear_sheet(factor_data, long_short=True, group_neutral=False)
```

5.6 Notes

- All tear sheets automatically apply consistent styling via the `@plotting.customize` decorator.
- GridFigure handles the layout complexity automatically; most workflows don't require direct use.
- The `by_group` parameter enables side-by-side or panel analysis when asset groups are present.
- For large datasets or many forward-return periods, tear sheets can take time to render; `create_summary_tear_sheet()` is faster for quick feedback.
- All plotting occurs within the tear sheet functions; call `plt.show()` or allow the Jupyter kernel to display results automatically.